

handle restarts if the Service is killed by the system prior to an explicit call to `stopService` or `stopSelf`.

The following snippet extends the previous code to show the skeleton code for overriding the `onStartCommand` handler. Note that it returns a value that controls how the system will respond if the Service is restarted after being killed by the run time.

```
@Override
    public int onStartCommand(Intent intent, int flags, int
startId) {
    // TODO Launch a background thread to do processing.
    return Service.START_STICKY;
}
```

Services are launched on the main Application thread, meaning that any processing done in the `onStartCommand` handler will happen on the main GUI thread. The standard pattern for implementing a Service is to create and run a new Thread from `onStartCommand` to perform the processing in the background and stop the Service when it's complete.

This pattern lets `onStartCommand` complete quickly, and lets you control the restart behavior using one of the following Service constants:

- `START_STICKY` describes the standard behaviour, which is similar to the way in which `onStart` was implemented prior to Android 2.0. If you return this value, `onStartCommand` will be called any time your Service restarts after being terminated by the run time. Note that on restart the intent parameter passed on to `onStartCommand` will be null.
- This mode is typically used for Services that handle their own states, and that are explicitly started and stopped as required (via `startService` and `stopService`). This includes Services that play music or handle other ongoing background tasks.
- `START_NOT_STICKY` is used for Services that are started to process specific actions or commands. Typically they'll use `stopSelf` to terminate once that command has been completed.
- Following termination by run time, Services set to this mode will restart only if there are pending start calls. If no `startService` calls have been made since the Service was terminated, the Service will be stopped without a call being made to `onStartCommand`.
- This mode is ideal for Services that handle specific requests, particularly regular processing such as updates or network polling. Rather than restarting the Service during a period of resource